TileGrad: a typed tile language over tinygrad UOps

Version 1 target specification

TileGrad contributors

July 2026

Abstract

TileGrad is a statically scheduled, Python-embedded language for tiled compute kernels. Programs expose tile shapes, memory placement, launch shape, and coarse scheduling intent while the compiler owns lane-level distribution, synchronization, and legal instruction selection. TileGrad lowers into tinygrad UOps and delegates backend code generation, compilation, and runtime execution to tinygrad.

This document specifies the intended stable Version 1 language. It is a north-star contract rather than a description of the current experimental implementation. A conforming implementation may initially use scalar or SIMT fallbacks for every tile operation. Optimized copy, matrix, and pipeline lowerings must preserve the same semantics.

Contents

1 Status and Conformance

The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** describe normative requirements. Text marked "non-normative" provides rationale or examples.

A *program* conforms when it uses only constructs defined by this specification and satisfies all static validity rules. An *implementation* conforms when it accepts every program required by its declared profile, rejects invalid programs before device execution, and preserves the observable semantics in this document.

Performance is not part of semantic conformance. In particular, vectorized loads, native matrix instructions, asynchronous transfers, physical pipeline overlap, occupancy, and instruction counts are not guaranteed unless an explicit capability is requested.

1.1 Design goals

- Make tiled kernels substantially easier to express than raw custom TINYGRAD UOps.
- Keep tile shape, memory placement, launch shape, and coarse scheduling visible in source and debug IR.
- Provide one correctness path whose meaning is independent of target instruction selection.
- Permit optimized lowering without changing source-level semantics.
- Keep the compiler boundary small: TILEGRAD owns the typed tile IR; TINYGRAD owns UOps, rendering, compilation, and runtime execution.

1.2 Non-goals

Version 1 does not standardize automatic schedule search, autotuning, arbitrary layout functions, TileLang or TVM compatibility, backend source injection, clusters, tensor memory, TMA, WGMMA, TCGEN05, user-managed asynchronous copy, wait groups, warp specialization, scans, atomics, or implicit autograd.

2 Compilation Model

A source program passes through the following conceptual stages:

Stage	Responsibility
Specialize	Bind compile-time constants and tensor shape constants.
Parse	Convert the supported Python subset into source-located typed IR.
Verify	Check types, shapes, effects, access modes, uniformity, and resource rules.
Normalize views	Rewrite every view to a base buffer, offset, shape, and strides.
Legalize layouts	Select a canonical logical-parallel and fragment ownership layout.
Plan effects	Insert or validate dependencies and synchronization for managed memory.
Legalize tile ops	Select scalar/SIMT lowering or an exact available capability.
Lower	Emit scalar, vector, range, memory, barrier, and optional WMMA UOps.
Verify UOps	Apply the relevant TINYGRAD UOp program specification.
Execute	Delegate rendering, compilation, launch, and storage to TINYGRAD.

Each stage **MUST** be inspectable. At minimum, implementations expose the parsed tile IR, verified tile IR, legalized tile IR, scalar/SIMT IR, and final UOps. A native operation selected during legalization must remain identifiable in debug output.

3 Python Frontend

The canonical frontend is a Python decorator named `@tilegrad.jit`. It has one explicit behavior: calling a decorated function specializes, compiles, caches, and executes a TILEGRAD program. An implementation **MUST NOT** infer a different eager or lazy API from the function body or return type.

3.1 Supported source structure

A decorated function consists of:

1. runtime tensor and scalar parameters;
2. keyword-only compile-time parameters annotated `T.Const`;
3. tensor shape declarations and output declarations;
4. exactly one `with T.Kernel(...)` launch region;
5. zero or more compile-time helper or macro expansions; and
6. a return of every declared output, preserving tuple order.

Each runtime tensor parameter must have exactly one annotated declaration of the following form before its first use:

```
name: T.Tensor[(shape...), dtype]
```

`T.const("M", ...)` declares named specialization symbols. Every such symbol must occur directly as one dimension in at least one input tensor declaration. At specialization, direct occurrences bind the symbol to the corresponding runtime dimension; repeated occurrences must agree. An unbound symbol or conflicting dimensions are specialization errors. Output declarations do not bind symbols.

A runtime scalar parameter has an annotation `T.Scalar[dtype]`. At the current TINYGRAD boundary it is represented by a dense rank-zero tensor input and loaded once inside the kernel. It cannot affect launch dimensions. Compile-time helper functions execute during specialization. Device macros must be marked `@T.macro`, are non-recursive, and are inlined into the typed IR before verification.

The device subset includes scalar assignments, tuple unpacking, calls to specified TILEGRAD operations, `for` over TILEGRAD loop constructs, structured `if`/`else`, a kernel launch `with` statement, and a final return. Python classes, recursion, generators, exceptions, arbitrary iteration, `while`, `break`, and `continue` are outside Version 1.

Ordinary Python control flow may inspect only compile-time values. A condition depending on a device value creates device control flow. A frontend **MUST NOT** coerce a device predicate to a Python boolean.

3.2 Canonical program

The following spelling is normative for the Version 1 decorator frontend.

```
@tilegrad.jit
def matmul(A, B, *,
           BM: T.Const[int] = 64,
           BN: T.Const[int] = 64,
           BK: T.Const[int] = 32):
  M, N, K = T.const("M", "N", "K")
  A: T.Tensor[(M, K), T.float16]
  B: T.Tensor[(K, N), T.float16]
  C = T.empty((M, N), T.float16)

  with T.Kernel(T.ceildiv(N, BN), T.ceildiv(M, BM),
                threads=128) as (bx, by):
    A_shared = T.shared((BM, BK), T.float16)
    B_shared = T.shared((BK, BN), T.float16)
    accum = T.fragment((BM, BN), T.float32)
    T.clear(accum)

    for ko in T.Pipelined(T.ceildiv(K, BK), stages=3):
      T.copy(A.tile((by*BM, ko*BK), (BM, BK)),
             A_shared, pad=0)
      T.copy(B.tile((ko*BK, bx*BN), (BK, BN)),
             B_shared, pad=0)
      T.mma(A_shared, B_shared, accum)

    T.copy(accum, C.tile((by*BM, bx*BN), (BM, BN)))
  return C
```

3.3 Normative API surface

Version 1 implementations must accept the following source constructs. Shape, dtype, and integer arguments below are typed expressions subject to the static rules in this document.

Source form	Result or effect
`T.const(*names)`	Declare specialization symbols; return one symbol for one name and a tuple in argument order for multiple names.
`x: T.Tensor[(shape...), dtype]`	Declare the type of runtime tensor parameter `x`.
`x: T.Scalar[dtype]`	Declare a runtime rank-zero scalar parameter.
`T.empty(shape, dtype)`	Declare and return a new global output tensor.
`T.shared(shape, dtype, sync="managed")`	Allocate workgroup shared storage with managed or manual synchronization.
`T.fragment(shape, dtype, policy="auto")`	Allocate a collective fragment using auto, row-major, or column-major policy.
`T.private(shape, dtype)`	Allocate private, statically indexed storage.
`V.tile(origin, shape)`	Create a possibly out-of-base tile view with the same rank as `V`.
`V.subview(origin, shape)`	Create a view that must be provably within the logical extent of `V`.
`V.transpose(order)`	Permute shape and strides by a rank permutation.
`V.reshape(shape)`	Create an equal-element-count view when contiguity is proven.
`T.Kernel(*grid, threads=...)`	Enter the single launch region and yield grid coordinates in dimension order.
`T.Serial(stop)` or `T.Serial(start, stop, step=1)`	Construct an ordered half-open loop.
`T.Unroll(stop)` or `T.Unroll(start, stop, step=1)`	Construct a compile-time-bounded loop requested to unroll.
`T.Reduce(stop)` or `T.Reduce(start, stop, step=1)`	Construct an ordered scalar recurrence loop represented by a reduction axis.

Source form	Result or effect
`T.Parallel(*extents, policy="auto")`	Construct a logical Cartesian parallel loop and yield one coordinate per extent.
`T.Pipelined(stop, stages=N)` or `T.Pipelined(start, stop, stages=N)`	Construct a unit-step loop with serial semantics and positive compile-time pipeline hint `N`.

The scalar and tile operation forms are:

Source form	Result or effect
`T.load(V, indices, mask=True, other=0)`	Perform the scalar load defined in Section 10; `indices` is a scalar for rank one and a tuple otherwise.
`T.store(V, indices, value, mask=True)`	Perform the scalar store defined in Section 10.
`V[indices]` and `V[indices] = value`	Abbreviate unmasked `T.load` and `T.store`; fragment indexing is legal only in a matching `T.Parallel`.
`T.fill(dst, value)` and `T.clear(dst)`	Fill a collective destination with a scalar, or with zero.
`T.copy(src, dst, pad=0)`	Perform equal-shape synchronous copy.
`T.mma(A, B, C, transpose_a=False, transpose_b=False, mode="auto")`	Accumulate a rank-two matrix product into `C`; mode is `auto`, `simt`, or a versioned capability name.
`T.reduce(src, axis, op, accumulator_dtype=None, keepdim=False, initial=None, nan_policy="propagate")`	Return a fresh fragment reduced by `sum`, `max`, or `min`; omitted accumulator dtype means the source dtype.
`T.reduce_sum`, `T.reduce_max`, `T.reduce_min`	Aliases fixing the corresponding `op` argument of `T.reduce`.
`T.where(predicate, x, y)` and `T.cast(x, dtype)`	Scalar or pointwise lifted fragment select and conversion.
`T.thread_id(dim)` and `T.thread_extent(dim)`	Return expert local-agent coordinates and extents.
`T.barrier()`	Perform the convergent workgroup barrier.

Arithmetic, comparison, logical, and math operations automatically lift to fragments and create the fragment-map nodes defined below. Scalar-to-fragment and compatible fragment-to-fragment broadcasting follows the Version 1 broadcast rule. Rebinding a fragment name or assigning its logical coordinates creates a new fragment version in typed IR.

4 Parameters, Shapes, and Outputs

4.1 Value categories

Category	Meaning	Permitted uses
Compile-time constant	Included in the specialization cache key.	Python control flow, tile and allocation shapes, workgroup shape, layouts, unrolling, and all device expressions.
Tensor shape constant	Inferred from a runtime tensor, then specialized.	Grid extents, output shapes, bounds, and device expressions.
Runtime scalar	Passed to the device program without specialization.	Device expressions, predicates, indices, and loop bounds; not launch, allocation, fragment, layout, unroll, or workgroup shapes.

Tensor rank and dtype are always specialization-time information. Version 1 specializes tensor dimensions by default. Implementations may later declare a shape-polymorphic extension, but it must not weaken the rules for static shared or fragment storage.

4.2 Outputs

`T.empty(shape, dtype)` declares a dense, contiguous global output. It does not allocate device-local storage inside the kernel. Frontend lowering creates an output specification and the host wrapper allocates the tensor before invoking TINYGRAD custom-kernel execution.

Every output **MUST** be returned exactly once. Only declared outputs may be returned. Output shape expressions may depend on input shape constants and compile-time constants. Multiple outputs are returned as a tuple in source order. Outputs must not alias inputs or one another. Scalar device results are represented as rank-zero or one-element output tensors, not Python values.

Runtime tensor inputs are read-only in the core profile. A future in-place extension must make read/write access explicit and must define aliasing.

5 Scalar Types and Expressions

Core scalar types are `bool`, signed and unsigned 8-, 16-, 32-, and 64-bit integers, `float16`, `bfloat16`, and `float32`. An implementation may expose additional TINYGRAD dtypes through declared capabilities.

Expressions are statically typed. Implicit conversion is limited to a Python literal converted to the unambiguous type required by its context. All other dtype changes require `cast`. Integer indices use the implementation's index type and must not be silently converted from floating point.

Class	Operations
Arithmetic	`+`, `-`, `*`, floating division, integer floor division, remainder, negation, minimum, and maximum.
Comparison	`<`, `<=`, `==`, `!=`, `>=`, and `>`; result dtype is `bool`.
Logical	elementwise and, or, not, and `select(predicate, a, b)`.
Conversion	numeric cast and bitcast where equal storage width is proven.
Math	absolute value, reciprocal, square root, `exp2`, `log2`, and sine.
Constants	finite numeric values, positive and negative infinity, and typed reduction identities.

`exp` and `log` may be frontend compositions over `exp2` and `log2`. Floating-point reassociation and reduction order are implementation-defined within the numerical contract in Section 15.

6 Buffers and Views

A buffer type is the tuple

$$(\text{dtype},\ \text{rank},\ \text{shape},\ \text{strides},\ \text{address space},\ \text{access}).$$

Core implementations support ranks zero through four. They may support higher ranks. External tensors are dense, non-overlapping, and contiguous. This restriction matches the current TINYGRAD custom-kernel boundary, which may materialize non-contiguous input views.

A view is

$$V = (B,\ o,\ \mathbf{s},\ \mathbf{d},\ n_B),$$

where B is a base buffer, o is an element offset, $\mathbf{s}$ is a logical shape, and $\mathbf{d}$ contains element strides. The base storage contains n_B elements. The address of logical coordinate $\mathbf{i}$ is

$$\operatorname{addr}(V, \mathbf{i}) = o + \sum_k i_k d_k.$$

A coordinate is in bounds exactly when

$$\operatorname{inbounds}(V, \mathbf{i}) \iff \left(\bigwedge_k 0 \le i_k < s_k \right) \wedge 0 \le \operatorname{addr}(V, \mathbf{i}) < n_B.$$

The storage extent is the element count of an external tensor or allocation. `tile(origin, shape)` may intentionally create a logical view extending beyond base storage; masked load, store, fill, and copy use the predicate above to define edge behavior.

Subview and transpose are view operations and do not copy. Reshape is valid only when contiguity can be proven. Writable zero-stride views, overlapping writable views, and negative strides are invalid in Version 1.

Two views overlap when their sets of valid base-storage addresses intersect. Copy between identical views is a no-op. Every other overlapping source and destination copy is invalid; Version 1 does not provide snapshot or `memmove` semantics. If disjointness cannot be proven, compilation must fail.

Negative coordinates are out of bounds; they do not wrap as Python indices. Bounds behavior is defined by the operation performing the access. There is no general pass that silently repairs arbitrary out-of-bounds shared or private accesses.

7 Memory Spaces

Space	Ownership	Shape	Purpose
Global	Invocation/tensor	Static rank; specialized dimensions	Inputs and outputs.
Shared	Workgroup	Compile-time constant	Cooperative staging and communication.
Fragment	Workgroup collective	Compile-time constant	Logically shaped values distributed across lanes.
Private	One local agent	Compile-time constant	Scalars and statically indexed register tuples.

Shared, fragment, and private allocations have lexical lifetime inside one launch region. Dynamic shared allocation is outside the core profile. Private tuple indexing must become constant after specialization and unrolling; otherwise compilation fails rather than silently spilling to another memory space.

8 Execution Model

A launch is a grid of independent workgroups. A grid has one to three positive specialization-time extents. A workgroup has one to three positive specialization-time local extents or one positive total thread count. No order or implicit synchronization exists between workgroups.

The values yielded by `T.Kernel` are grid coordinates. Expert code uses `T.thread_id(dim)` for a local coordinate and `T.thread_extent(dim)` for its extent, where `dim` is 0, 1, or 2.

8.1 Loop forms

Form	Semantics
Serial	Ordered iterations over a half-open integer interval.
Unroll	Serial semantics with a compile-time request to expand iterations.
Reduce	Ordered recurrence semantics that the lowerer may reassociate when allowed by the numerical contract.
Parallel	An unordered logical Cartesian iteration space; every point executes exactly once.
Pipelined	Serial logical semantics with a pipeline-depth optimization hint.

8.2 Logical parallel mapping

`T.Parallel` is not an alias for a hardware local axis. It defines an unordered logical iteration space distributed over the current workgroup. The compiler selects from a bounded set of canonical policies. Let the iteration shape be $(e_0, \ldots, e_{r-1})$, let P be the workgroup size, and let $\ell \in [0, P)$ be the row-major linear local-agent identifier.

- `row_major`: use $q_R(\mathbf{i}) = \sum_k i_k \prod_{j>k} e_j$; owner $q_R \bmod P$, private slot $\lfloor q_R/P \rfloor$;
- `column_major`: use $q_C(\mathbf{i}) = \sum_k i_k \prod_{j<k} e_j$; owner $q_C \bmod P$, private slot $\lfloor q_C/P \rfloor$; and
- an internal exact mapping required by a selected native matrix capability.

An agent executes the coordinates whose linearized value is $\ell + tP$ for nonnegative t while that value is less than $\prod_k e_k$. This rule covers iteration spaces smaller than, equal to, or larger than the workgroup.

The source may request a canonical policy or `auto`. `auto` may select only from the specified finite set. Unsupported or ambiguous composition must fail deterministically. The selected policy **MUST** appear in legalized debug IR.

If explicit thread identifiers affect the same memory operations as a logical parallel loop, the loop policy must be explicit. An implementation must not make observable program behavior depend on an unreported automatic mapping.

9 Fragments

A fragment is a statically shaped, workgroup-collective logical value. Its physical register ownership is not observable source state. Legalization assigns each logical coordinate to a canonical lane and private slot, or to an exact native matrix fragment layout.

Every fragment carries a resolved mapping policy. A logical parallel region "matches" a fragment when its extents equal the fragment shape and its resolved policy is identical. Access to coordinate $\mathbf{i}$ in that region is executed only by the owner defined by the mapping formula above. All operations produce a new logical fragment version; source versions are read before destination writes, including syntactically in-place elementwise forms.

Fragments support:

- fill and clear;
- synchronous copy to and from compatible views;
- elementwise unary, binary, comparison, select, and cast operations;
- rank-compatible scalar and row/column broadcasting;
- axis reductions; and
- matrix multiply-accumulate.

Fragment coordinates may be read or written only by a tile operation or inside a matching logical parallel region. Arbitrary fragment indexing by a runtime thread identifier is invalid. SIMT fallback must implement the same logical coordinate ownership; it must not replicate the complete fragment in every thread.

9.1 Fragment elementwise algebra

$\textsc{MapUnary}(f, X)$, $\textsc{MapBinary}(f, X, Y)$, $\textsc{Select}(P, X, Y)$, and $\textsc{Cast}(T, X)$ apply pointwise and produce a fresh fragment. Operands use right-aligned broadcasting: for each aligned axis, extents must be equal or one; a missing leading axis and an extent-one axis repeat the same source coordinate. The result extent is the maximum of the aligned extents. Dtypes follow the scalar expression rules.

Scalar operands are rank-zero fragments for broadcasting purposes. Row and column broadcasting are not special cases: shapes $(M, 1)$ and $(1, N)$ follow the same rule. Fragment-to-fragment copy requires equal shapes; use an elementwise broadcast operation when expansion is intended.

10 Loads, Stores, and Effects

The scalar memory operations are:

$$\textsc{Load}(V, \mathbf{i}, m, a) = \begin{cases} V[\mathbf{i}] & m \wedge \text{inbounds}(V, \mathbf{i}) \\ a & \text{otherwise,} \end{cases}$$

$$\textsc{Store}(V, \mathbf{i}, x, m): \quad V[\mathbf{i}] \leftarrow x \quad \text{iff } m \wedge \text{inbounds}(V, \mathbf{i}).$$

The mask defaults to true. The alternate load value defaults to typed zero. A value is explicitly cast before storage when source and destination dtypes differ.

Effects to the same logical location are ordered by source order unless they occur in an unordered parallel region. Two unordered writes, or an unordered read and write without a defined collective or synchronization relation, form a data race and are invalid. Version 1 has no atomic escape from this rule.

11 Tile Operations

Tile operations are collective statements with explicit logical read and write regions. They must be reached uniformly by every participating local agent. A tile operation in thread-divergent control flow is invalid unless the operation explicitly defines predicated collective participation.

11.1 Fill and clear

$$\textsc{Fill}(D, x): \qquad D[\mathbf{i}] \leftarrow \mathrm{cast}_{D.\mathrm{dtype}}(x)$$

for every valid logical coordinate $\mathbf{i}$ in D. $\textsc{Clear}(D)$ is exactly $\textsc{Fill}(D, 0)$. Fill never writes outside the destination view.

11.2 Synchronous copy

For equal-shaped logical regions S and D,

$$\textsc{Copy}(S, D, p): \qquad D[\mathbf{i}] \leftarrow \begin{cases} \mathrm{cast}_{D.\mathrm{dtype}}(S[\mathbf{i}]) & \mathrm{inbounds}(S, \mathbf{i}) \\ \mathrm{cast}_{D.\mathrm{dtype}}(p) & \text{otherwise} \end{cases}$$

when $D[\mathbf{i}]$ is in bounds. An out-of-bounds destination coordinate performs no write.

Source and destination shapes must be equal. Copy does not perform implicit broadcasting and does not infer a tile extent from an unrelated operand.

Copy has safe-to-consume semantics: a subsequent dependent high-level tile operation observes the completed destination values. The compiler inserts or elides transfer completion and workgroup visibility operations as necessary. This guarantee includes ordering before managed shared storage is reused.

The compiler may lower copy with scalar, vector, SIMT-cooperative, or target-specific transfer instructions, but instruction choice is not observable. A regular copy never exposes a partially completed asynchronous transfer.

11.3 Matrix multiply-accumulate

For rank-two operands, Mma has the mathematical semantics

$$C \leftarrow C + \mathrm{cast}_{T_{acc}}(\mathrm{op}_A(A)) \times \mathrm{cast}_{T_{acc}}(\mathrm{op}_B(B)),$$

where each op is identity or transpose. Shapes must satisfy

$$A : (M, K), \qquad B : (K, N), \qquad C : (M, N)$$

after transpose. The operation does not implicitly clear C.

Core implementations support float32 SIMT MMA and float16 or bfloat16 inputs with float32 accumulation. Additional dtype pairs are capabilities.

The selection mode is one of:

- `auto`: use any conforming SIMT or native implementation;
- `simt`: do not select a native matrix instruction; or
- an exact native capability: require its atom shape, dtypes, participant count, and layout, failing compilation if unavailable.

An exact request must never silently fall back. `auto` must preserve the same logical fragment and numerical contract whichever path is selected.

An exact native capability declares an atom (M_a, N_a, K_a). A logical MMA may use that capability only when M, N, and K are integer multiples of the corresponding atom dimensions. The compiler partitions M and N into disjoint atoms and visits K atoms in increasing logical order. A global edge is represented by already padded input tiles and suppressed output stores; an exact native MMA itself never accesses a partial atom. If these rules cannot be met, an exact request fails and `auto` uses another legal capability or SIMT.

11.4 Axis reduction

$\textsc{Reduce}(X, a, f, T_{acc}, k)$ reduces axis a of X, using operation f, accumulator dtype T_{acc}, and keep-dimension flag k. Version 1 operations are sum, maximum, and minimum.

The result shape removes axis a when k is false and replaces it with one when k is true. Reduction starts from the operation identity unless an explicit initial value is provided. Values are converted to the accumulator dtype before combination.

Operation	Identity	NaN policy
sum	typed zero	NaNs propagate
maximum	negative infinity or integer minimum	explicit propagate or ignore
minimum	positive infinity or integer maximum	explicit propagate or ignore

Maximum and minimum default to `nan_policy="propagate"`. An implementation may emulate this policy when a native instruction differs. Reduction order is otherwise implementation-defined.

12 Synchronization and Uniformity

Managed high-level operations use effect analysis to establish required completion and visibility. The implementation must conservatively preserve:

- read-after-write dependencies;
- write-after-read dependencies before storage reuse;
- write-after-write source order; and
- loop-carried dependencies.

A portable workgroup barrier waits for all local agents and establishes a happens-before relation from prior shared writes to subsequent shared reads by participants. Every barrier must be reached convergently by the same participant set. A barrier under a thread-varying conditional is invalid.

Expert code may request `T.shared(shape, dtype, sync="manual")` instead of the default `sync="managed"`. Managed and manual synchronization must not be mixed for one allocation. Raw shared access through explicit thread identifiers is permitted only for manually synchronized allocations; the author is then responsible for barriers and race freedom. `T.barrier()` is the source operation for the portable workgroup barrier. High-level collective copy, fill, reduction, and MMA do not accept manually synchronized shared operands in the core profile.

There is no grid-wide barrier in the core profile.

13 Pipelined Loops

A pipelined loop has exactly the observable behavior of dependency-respecting serial iterations. Each logical iteration executes once, each effect occurs once, and no consumer observes uninitialized or prematurely overwritten storage.

The stage count is a positive compile-time optimization hint. It does not guarantee physical overlap, a number of in-flight iterations, a particular instruction, a particular number of storage versions, or improved performance. Executing the loop serially is semantically conforming.

An optimized implementation may reorder operations, version shared storage, and use asynchronous target instructions internally. It must preserve the synchronous copy contract and all loop-carried dependencies. Manual stage/order arrays, transfer tokens, wait groups, and hardware barrier parity are not Version 1 source constructs.

14 Structured Control Flow

Structured `if`/`else` is permitted over a boolean scalar. Expression-level selection should be used for simple predication. Loads and stores may carry explicit masks.

A conditional is *workgroup-uniform* when every local agent evaluates the same branch. Collective tile operations and barriers may appear only in uniform control flow. The compiler must prove uniformity from compile-time values, grid coordinates, and other known-uniform values, or reject the program. Local thread identifiers and values loaded under thread-varying indices are conservatively non-uniform. Version 1 has no user assertion that overrides uniformity analysis.

15 Numerical Contract

Integer addition, subtraction, and multiplication are modulo 2^w for a w-bit dtype. Comparisons use the dtype's signedness. Floor division and remainder follow mathematical floor division. Defined programs have a nonzero divisor and shift counts in $[0, w)$. Statically invalid cases must be rejected; behavior outside the defined domain is not covered by conformance.

Floating-point elementwise operations follow the selected TINYGRAD dtype and backend within ordinary IEEE-754 expectations. The compiler may fuse a multiply and add, reassociate a reduction, or use a native matrix instruction. Consequently, semantic equality is not generally bitwise equality.

Conformance vectors use finite inputs in $[-1, 1]$ unless exceptional values are the subject of the test. Integer, boolean, movement, mask, and correctly rounded cast tests are exact. Default floating tolerances are:

Class	**Absolute tolerance**	**Relative tolerance**
float32 elementwise	10^{-6}	10^{-6}
float32 reduction/MMA, length R	$10^{-5} \max(1, R)$	10^{-5}
float16/bfloat16 output	$2 \cdot 10^{-2}$	$2 \cdot 10^{-2}$
FlashAttention output	$2 \cdot 10^{-2}$	$2 \cdot 10^{-2}$

A native and SIMT MMA must implement the same mathematical accumulation dtype, but may differ in rounding within these limits. NaNs compare classification only because payload and sign may be canonicalized. Infinity and signed-zero tests compare classification and sign exactly. Selected reduction NaN-policy tests compare whether a NaN is present. Masked-off values must not introduce exceptional values beyond the specified alternate or reduction identity.

16 Capabilities and Failures

Capabilities describe optional legal lowerings, not alternate language semantics. A capability record includes a stable name, target predicate, operand dtypes, accumulator dtype, atom shape, participant count, operand scope requirements, and internal fragment layout.

Capability names are namespaced and versioned, for example `tinygrad.cuda.m16n8k16.f16.f32.v1`. The implementation exposes `tilegrad.capabilities(target)` to enumerate records. Each record also contains the exact TINYGRAD WMMA argument schema expected by the compatibility adapter. Renaming or changing the meaning of a record requires a new versioned name.

The compiler must distinguish:

- an invalid TILEGRAD program;
- a valid program unavailable on the selected profile or target;
- a failure to produce valid TINYGRAD UOps; and
- a backend compilation or runtime failure.

Diagnostics must identify the source construct, expected contract, observed types or shapes, and selected target where relevant. Unsupported exact native requests fail during capability legalization, before UOp construction.

17 Boundary with tinygrad

TILEGRAD owns the frontend, typed IR, specialization, views, effects, uniformity, canonical logical layouts, synchronization planning, tile legalization, SIMT algorithms, and exact native capability adaptation.

TINYGRAD owns UOp validation and rewriting after legalization, scalar and vector rendering, device-specific code generation, compilation, launch, storage integration, and outer tensor-graph scheduling.

TILEGRAD must not depend on TINYGRAD optimization to recover missing memory dependencies, fragment ownership, or synchronization. Integration occurs through one versioned compatibility layer and a tested TINYGRAD revision range.

The core profile has no implicit gradient. Differentiation through a TILEGRAD kernel must fail clearly unless an explicit backward kernel or VJP is registered.

18 Conformance Profiles

18.1 Core profile

The core profile requires:

- decorator parsing and specialization;

- dense contiguous tensor inputs and one or more outputs;
- one- through three-dimensional grids and workgroups;
- scalar expressions, views, masks, and structured conditionals;
- managed shared storage and canonical logical parallel mapping;
- fragments, fill, clear, copy, sum/max/min reduction, and SIMT MMA;
- serial execution of a semantically pipelined loop when necessary; and
- complete debug IR and final UOp validation.

18.2 Accelerated profile

The accelerated profile is additive. It may provide vectorized copy, native matrix capabilities, real software pipelining, asynchronous instructions hidden behind synchronous operations, and additional dtypes. A backend may implement some accelerated capabilities without implementing all of them.

19 Required Conformance Programs

A release claiming core conformance must test at least:

1. exact and masked one- through four-dimensional copy;
2. internal strided subviews and transpose;
3. source padding, destination suppression, cross-dtype copy, and rejected overlapping copies;
4. one- through three-dimensional grids, canonical ownership mappings, logical parallel loops, and rejected runtime-scalar launch extents;
5. uniform and rejected divergent collective operations;
6. shared-memory producer/consumer and reuse dependencies;
7. fragment unary, binary, broadcast, select, and cast operations;
8. sum, maximum, and minimum reductions, including non-power-of-two tails;
9. float32 SIMT GEMM with M, N, and K edge tiles;
10. float16 and bfloat16 input GEMM with float32 accumulation;
11. unavailable exact-native capability failures;
12. multiple output allocation and return ordering; and
13. small causal and non-causal FlashAttention forward kernels using stable online softmax, edge sequence lengths, reduction, `exp2`, and two MMA operations.

The same mathematical cases should run through SIMT and every available native capability. Negative tests are part of conformance, not optional robustness tests.

A Normative IR Summary

The exact in-memory Python classes are not normative. A conforming typed IR must represent at least the following semantic nodes.

Node	Operands	Meaning
Program	parameters, outputs, launch	One specialized callable program.
Buffer parameter	type, shape, access	Dense external tensor storage.
Scalar parameter	dtype, category	Compile-time or runtime scalar.
Output specification	shape, dtype, order	Host-allocated returned tensor.
View	base, offset, shape, strides, storage extent	Logical region sharing base storage.

Node	Operands	Meaning
Allocate shared	shape, dtype, sync mode	Workgroup-visible storage.
Allocate fragment	shape, dtype	Collective logical register tile.
Allocate private	shape, dtype	Per-agent statically indexed storage.
Kernel launch	grid, workgroup	Grid of independent workgroups.
Serial loop	start, stop, step, body	Ordered iteration.
Unroll loop	static interval, body	Ordered iteration requested to expand.
Reduce loop	interval, recurrence	Scalar recurrence.
Parallel loop	extents, policy, body	Unordered logical iteration.
Pipelined loop	interval, stages, body	Serial semantics with pipeline hint.
Thread index	dimension	Explicit local-agent coordinate or extent.
If	predicate, branches	Structured control flow.
Load	view, coordinate, mask, alternate	Predicated scalar read.
Store	view, coordinate, value, mask	Predicated scalar write.
Fill	destination, scalar	Collective typed fill.
Copy	source, destination, pad	Collective synchronous region copy.
Fragment map	operation, sources, broadcast shape	Pointwise collective fragment operation.
MMA	A, B, C, transpose, mode	Collective accumulating matrix multiply.
Reduce	source, axis, op, dtype, keepdim	Collective axis reduction.
Barrier	memory scope	Convergent workgroup synchronization.

B Non-normative Implementation Milestones

The intended implementation order is:

1. typed parameters, outputs, scalar expressions, and normalized views;
2. the decorator parser and source-located diagnostics;
3. canonical logical parallel mapping and fragment ownership;
4. first-class fill and copy with conservative synchronization planning;
5. fragment elementwise algebra and axis reductions;
6. mixed-precision SIMT MMA and the FlashAttention conformance kernel;
7. one exact native MMA capability adapted to TINYGRAD WMMA; and
8. optimized pipeline rewriting after shared-memory effects are stable.

The current builder frontend may remain as an internal construction and testing tool, but the Version 1 public contract is the decorated typed language defined above.